BeaconKit 아키텍처 분석 - 03. CometBFT 통합 및 ABCI 인터페이스

1. CometBFT 통합 개요

BeaconKit은 **CometBFT (Tendermint)**를 합의 엔진으로 사용하며, **ABCI++ (Application Blockchain Interface Plus Plus)** 프로토콜을 통해 통합됩니다. 이는 이더리움의 LMD-GHOST + Casper FFG 대신 검증된 Byzantine Fault Tolerant 합의 알고리즘을 사용하는 핵심적인 설계 결정입니다.

1.1 핵심 특징

- **Byzantine Fault Tolerance**: 최대 1/3 악의적 검증자 허용
- **즉시 완결성**: 블록이 커밋되면 즉시 최종 확정
- **라운드 기반 합의**: 타임아웃 시 새로운 라운드로 진행
- **순차적 높이**: 빈 블록 없이 연속적인 블록 높이

1.2 CometBFT Service 구조

```
type Service struct {
node *node.Node
nodeAddress cmtcrypto.Address
// 합의 파라미터 (블록체인 상태의 일부)
cmtConsensusParams *cmttypes.ConsensusParams
// 노드별 설정 (config.toml에서 로드)
```

```
cmtCfg *cmtcfg.Config

// 비콘 체인 컴포넌트들

Blockchain blockchain.BlockchainI

BlockBuilder validator.BlockBuilderI

// 상태 캐시 및 관리

cachedStates cache.States

sm *statem.Manager

// 컨텍스트 (CometBFT 제한사항으로 인한 워크어라운드)

ctx context.Context

}
```

2. ABCI++ 메서드 구현

BeaconKit은 ABCI++ 인터페이스의 다음 메서드들을 구현합니다:

2.1 Info

```
func (s *Service) Info(context.Context, *cmtabci.InfoRequest) (*cmtabci.InfoResp
```

목적: 애플리케이션 정보 및 마지막 블록 상태 반환

구현:

```
lastCommitID := s.sm.GetCommitMultiStore().LastCommitID()

appVersion := initialAppVersion
```

```
if  lastCommitID.Version > 0 {
appVersion, err = s.appVersion()
}


return &cmtabci.InfoResponse{
Data: AppName, // "beacond"
Version: sdkversion.Version, // SDK 버전
AppVersion: appVersion, // 앱 버전
LastBlockHeight: lastCommitID.Version,
LastBlockAppHash: lastCommitID.Hash,
}
```

2.2 InitChain

```
func (s *Service) InitChain(_ context.Context, req *cmtabci.InitChainRequest) (*
```

목적: 제네시스 블록 초기화

호출 시점: 체인 시작 시 한 번만

2.3 PrepareProposal

```
func (s *Service) PrepareProposal(_ context.Context, req *cmtabci.PrepareProposa
```

목적: 새로운 블록 제안 준비

호출 대상: 현재 라운드의 제안자 노드만

호출 빈도: 라운드당 한 번 (타임아웃 시 새 라운드에서 재호출 가능)

주요 흐름:

1. 컨텍스트 유효성 검사
2. `ValidatorService.BuildBlockAndSidecars()` 호출
3. 실행 엔진과 페이로드 빌딩 조정
4. 블록 및 블롭 사이드카 조립
5. CometBFT에 제안 반환

2.4 ProcessProposal

```
func (s *Service) ProcessProposal(_ context.Context, req *cmtabci.ProcessProposa
```

목적: 제안된 블록 검증

호출 대상: 모든 노드 (제안자 포함)

반환값: ACCEPT 또는 REJECT

핵심 구현 (`beacon/blockchain/process_proposal.go`):

```
func (s *Service) ProcessProposal(
ctx sdk.Context,
req *cmtabci.ProcessProposalRequest,
thisNodeAddress []byte,
) (transition.ValidatorUpdates, error) {
// STEP 1: 블록 및 블롭 디코딩
```

```
signedBlk, sidecars, err := s.ParseBeaconBlock(req)

if  err != nil {

return  nil, fmt.Errorf("failed to decode block and blobs: %w", err)

}

blk := signedBlk.GetBeaconBlock()

// STEP 2: 포크 버전 검증

forkVersion := s.chainSpec.ActiveForkVersionForTimestamp(math.U64(req.GetTime().

blkVersion := s.chainSpec.ActiveForkVersionForTimestamp(blk.GetTimestamp())

if !version.Equals(blkVersion, forkVersion) {

return  nil, fmt.Errorf("version mismatch: %w", ErrVersionMismatch)

}

// STEP 3: 블롭 커밋먼트 수 검증

blobKzgCommitments := blk.GetBody().GetBlobKzgCommitments()

if  len(blobKzgCommitments) != len(sidecars) {

return  nil, fmt.Errorf("sidecar commitment mismatch: %w", ErrSidecarCommitmentM

}

// STEP 4: 블록 구조 검증

// STEP 5: 블롭 KZG 증명 검증

// STEP 6: 실행 엔진 검증 (newPayload)

// STEP 7: 상태 전환 검증

}
```

검증 단계:

1. **구조적 검증**: 블록 형식, 서명, 필드 유효성
2. **포크 버전 일치**: CometBFT 시간과 블록 시간의 포크 버전 동일성
3. **블롭 검증**: KZG 커밋먼트와 사이드카 수 일치
4. **실행 페이로드 검증**: `newPayload` 호출로 EL 검증
5. **상태 전환 검증**: 비콘 체인 상태 머신 규칙 적용

2.5 FinalizeBlock

```
func (s *Service) FinalizeBlock(_ context.Context, req *cmtabci.FinalizeBlockReq
```

목적: 합의된 블록의 최종 실행 및 상태 커밋

호출 시점: 2/3+ 검증자가 블록에 합의한 후

호출 빈도: 높이당 정확히 한 번

핵심 구현 (`beacon/blockchain/finalize_block.go`):

```
func (s *Service) FinalizeBlock(
ctx sdk.Context,
req *cmtabci.FinalizeBlockRequest,
) (transition.ValidatorUpdates, error) {
// STEP 1: 블록 및 블롭 디코딩
signedBlk, blobs, err := s.ParseBeaconBlock(req)
// STEP 2: 시작 시 강제 동기화 (한 번만)
s.forceStartupSyncOnce.Do(func() {
parentProposerPubkey, err := st.ParentProposerPubkey(blk.GetTimestamp())
```

```
if  err == nil {
err = s.forceSyncUponFinalize(ctx, blk, parentProposerPubkey)
}
})
// STEP 3: 사이드카 최종화 (블록이 사이드카 가용성 확인)
if  err = s.FinalizeSidecars(ctx, req.SyncingToHeight, blk, blobs);  err != nil
return  nil, fmt.Errorf("failed finalizing sidecars: %w", err)
}
// STEP 4: 블록 최종화
consensusBlk := types.NewConsensusBlock(blk, req.GetProposerAddress(), req.GetTi
valUpdates, err := s.finalizeBeaconBlock(ctx, st, consensusBlk)
// STEP 5: 후처리 작업
return  valUpdates, s.PostFinalizeBlockOps(ctx, blk)
}
```

최종화 단계:

1. **블록 디코딩**: 제안과 동일한 블록 파싱
2. **시작 동기화**: 첫 블록에서 실행 엔진 헤드 동기화
3. **사이드카 저장**: DA 기간 내 블롭들을 AvailabilityStore에 저장
4. **상태 전환 실행**: 실제 비콘 상태 업데이트
5. **후처리**: FCU 업데이트, 메트릭 기록

2.6 Commit

```go
func (s *Service) Commit(_ context.Context, req *cmtabci.CommitRequest) (*cmtabc
```

목적: 상태 변경사항을 디스크에 영구 저장

반환값: 새로운 앱 해시 (상태 루트)

3. 블록 높이 생명주기

3.1 CometBFT 라운드 기반 합의

CometBFT는 각 높이에서 **라운드 기반 합의**를 수행합니다:

```
높이 N, 라운드 0: 제안자 A 실패/타임아웃
높이 N, 라운드 1: 제안자 B 시도
높이 N, 라운드 2: 제안자 C 성공 → 높이 N+1로 진행
```

3.2 ABCI 메서드 호출 순서

정상적인 블록 생성 시:

```
1. PrepareProposal (제안자만)
2. ProcessProposal (모든 노드)
3. 합의 투표 라운드들
4. FinalizeBlock (모든 노드)
5. Commit (모든 노드)
```

라운드 실패 시:

```
1. PrepareProposal (라운드 0 제안자)

2. ProcessProposal → REJECT 또는 타임아웃

3. PrepareProposal (라운드 1 제안자)

4. ProcessProposal → ACCEPT

5. 투표 및 FinalizeBlock
```

3.3 상태 관리

- **ProcessProposal**: 상태 변경을 검증하지만 커밋하지 않음
- **FinalizeBlock**: 실제 상태 변경 수행
- **Commit**: 변경사항을 디스크에 영구 저장

4. 타이밍 및 성능 고려사항

4.1 CometBFT 타임아웃 설정

```
# config.toml

[consensus]

timeout_commit = "500ms" # 최소 강제 (커밋 후 대기)

timeout_propose = "2000ms" # 최대 강제 (초기 제안 타임아웃)

timeout_prevote = "2000ms" # 최대 강제 (프리보트 수집)

timeout_precommit = "2000ms" # 최대 강제 (프리커밋 수집)
```

4.2 블록 생성 성능 목표

- **PrepareProposal**: ~1초 내 완료
- **ProcessProposal**: 빠른 검증 (구조적 검증 우선)
- **FinalizeBlock**: 결정론적 실행

4.3 실행 엔진 통합 지연

- **페이로드 빌딩**: `forkchoiceUpdate → getPayload` 체인
- **검증**: `newPayload` 호출
- **최종화**: 후속 `forkchoiceUpdate`

5. 오류 처리 및 복구

5.1 컨텍스트 취소 처리

```
if s.ctx.Err() != nil {
// 컨텍스트가 취소되면 시스템 종료 중
return nil, s.ctx.Err()
}
```

중요: CometBFT는 현재 적절한 컨텍스트 전파를 지원하지 않아 워크어라운드 필요

5.2 제안 실패 처리

- **PrepareProposal 실패**: 빈 제안 반환 (안전)
- **ProcessProposal 실패**: REJECT 반환 → 새 라운드
- **FinalizeBlock 실패**: 패닉 (합의 실패)

5.3 실행 엔진 오류

- **연결 실패**: 재시도 로직 (지수 백오프)
- **페이로드 오류**: 제안 실패 → 새 라운드
- **검증 실패**: 블록 거부

6. 트랜잭션 구조

BeaconKit은 CometBFT 트랜잭션을 다음과 같이 구성합니다:

```
const (
BeaconBlockTxIndex  uint = 0  // 비콘 블록 (필수)
BlobSidecarsTxIndex  uint = 1  // 블롭 사이드카 (선택적)
MaxConsensusTxsCount = 2  // 최대 트랜잭션 수
)
```

6.1 트랜잭션 파싱

```
func (s *Service) ParseBeaconBlock(req  interface{}) (*ctypes.SignedBeaconBlock,
// 1. 비콘 블록 디코딩 (인덱스 0)
// 2. 블롭 사이드카 디코딩 (인덱스 1, 선택적)
// 3. 포크 버전에 따른 타입 변환
}
```

7. 캐시 및 최적화

7.1 상태 캐시

```go
type Service struct {
// 성공적으로 검증된 블록들의 사후 상태를 메모리에 추적
// 재실행 없이 최종화 가능
cachedStates cache.States
}
```

7.2 최적화 전략

- **병렬 검증**: 블롭 KZG 증명 병렬 처리
- **조기 실패**: 구조적 검증을 먼저 수행
- **상태 캐싱**: ProcessProposal에서 검증된 상태 재사용

8. 모니터링 및 메트릭

8.1 성능 메트릭

- **블록 생성 시간**: PrepareProposal 소요 시간
- **검증 시간**: ProcessProposal 소요 시간
- **상태 전환 시간**: FinalizeBlock 소요 시간

8.2 오류 메트릭

- **제안 실패율**: 실패한 PrepareProposal 비율
- **검증 실패율**: REJECT된 ProcessProposal 비율
- **라운드 수**: 합의까지 필요한 라운드 수

9. 설계 원칙

9.1 안전성 우선

- 불완전한 데이터로 합의하지 않음
- 컨텍스트 취소 시 즉시 중단
- 오류 시 패닉보다는 안전한 실패

9.2 결정론적 실행

- 모든 노드에서 동일한 상태 전환
- 비결정적 요소 제거
- 재현 가능한 블록 처리

9.3 성능 최적화

- 병렬 처리 가능한 부분 식별
- 조기 실패로 불필요한 계산 방지
- 캐시를 통한 중복 계산 제거

이 ABCI++ 통합은 BeaconKit이 검증된 BFT 합의 알고리즘의 이점을 활용하면서도 이더리움 호환성을 유지할 수 있게 해주는 핵심 아키텍처입니다. 다음 문서에서는 블록 생성 프로세스를 더 자세히 살펴보겠습니다.